

4.1 Discrete-Time Fourier Series (DTFS)

So far, when we talk about vectors in Cartesian spaces ($\mathbb{R}^n$ or $\mathbb{C}^n$), we tend to use the standard basis, which simply consists of the vectors where one vector element is 1 and the rest are 0:

$$\left\{ \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \dots, \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix} \right\} \quad (1)$$

This basis is natural to us because it corresponds directly to our visual representation of vectors (each element of the vector represents how far from 0 the vector is in that specific component's direction).

However, we have learned that the only requirements for a basis are to span the desired vector space and to be linearly independent, so a vector space often has a limitless number of possible bases. In this note, we will explore a basis derived from complex exponentials (usually called the **Fourier basis**), which will lead to the **Discrete-Time Fourier Series (DTFS)**, an important topic in signal processing.

You may ask: why use a basis other than the standard basis if the standard basis is the most intuitive to understand? In this context, we will be working with discrete-time signals, defined below, which can be thought of as data. The important point here is that different bases emphasize different characteristics of the data, which can help in better understanding the data itself. In the case of the complex exponentials basis, the basis emphasizes how quickly the values of the vector change from component to component, or more formally the *frequency* of the data.

4.1.1 Vector Representation of Periodic Discrete-Time Signals

The DTFS concept is usually discussed in the context of discrete-time signals, which are distinct from the traditional Cartesian vectors we usually work with. However, periodic discrete-time signals specifically can be represented by finite length Cartesian vectors through a *one-to-one* mapping.

First, let's understand what we mean by discrete-time signals.

Definition 4.1 (Discrete-Time Signal): Discrete-time signals x are ordered sequences of values indexed by discrete integers n (called samples) such that $x[n]$ is the value of the signal at that time, often represented using stem plots:

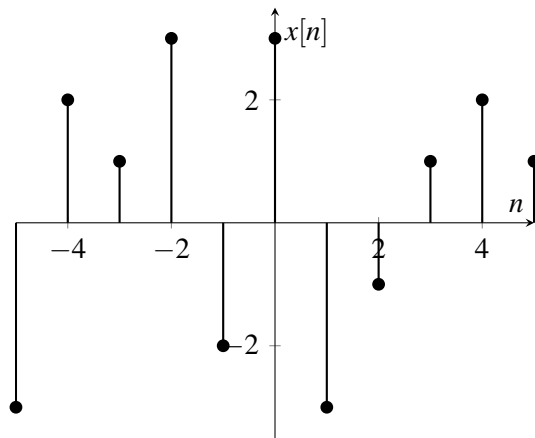


Figure 1: Stem Plot for a Discrete-Time Signal

If we wanted to describe a general discrete-time signal as a vector, it would take an infinite length vector since the sample indices vary from $n = -\infty$ to $n = \infty$.

However, some signals are periodic, which means the signal repeats after a certain number of samples. To be specific, a signal is p -periodic if the signal values repeat every p samples. Mathematically,

$$x[n] = x[n + p], \quad \forall n \in \mathbb{Z} \quad (2)$$

For example, here is the stem plot for a 3-periodic discrete-time signal:

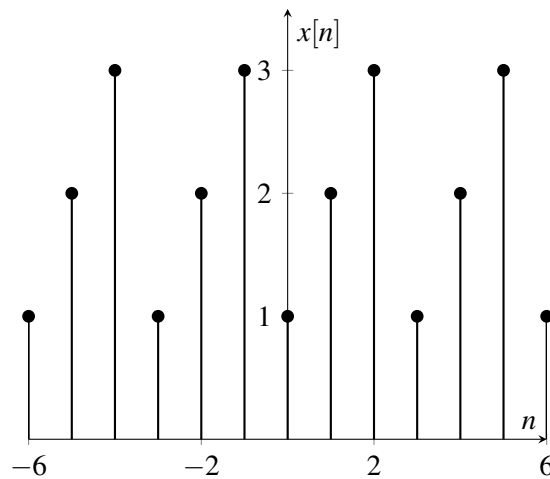


Figure 2: Stem Plot for a 3-Periodic Discrete-Time Signal

For a discrete-time signal x that is p -periodic, there are only p distinct values due to the periodicity. Thus, we can

represent such $x[n]$ using a p -dimensional vector space (usually $\mathbb{R}^p$ or $\mathbb{C}^p$):

$$\mathbf{x} = \begin{bmatrix} x[0] \\ x[1] \\ \vdots \\ x[p-1] \end{bmatrix} \quad (3)$$

For example, the signal represented in Figure ?? could be represented by the following 3D vector:

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \quad (4)$$

Note: We choose to have the first value of the vector be $x[0]$ by convention. By doing this, there is a **one-to-one** mapping from a periodic discrete-time signal to its vector representation. Thus, we will use linear algebra to process these vector representations (particularly change of basis, more on this later in the note and the course) to understand the underlying discrete-time signal, such as with the DTFS.

Definition 4.2 (Fundamental Period and Frequency):

In Figure ??, you may notice that the discrete-time signal is 3-periodic, but it is also 6-periodic, 9-periodic, 12-periodic, etc. But, at the same time, we know that calling it 3-periodic makes the most sense.

To formalize this, the **fundamental period** p of a discrete-time signal x is the *minimum* value of p for which $x[n]$ is periodic ($x[n] = x[n+p] \quad \forall n \in \mathbb{Z}$).

Correspondingly, the **fundamental angular frequency** is defined based on the fundamental period:

$$\omega_0 = \frac{2\pi}{p} \quad (5)$$

Note that angular frequency is in $\frac{\text{rad}}{\text{s}}$. Often, people will also use frequency in $\frac{\text{cycles}}{\text{s}}$, or Hertz (Hz):

$$f_0 = \frac{1}{p} \quad (6)$$

The relationship between the two frequencies is:

$$\omega_0 = 2\pi f_0 \quad (7)$$

4.1.2 DTFS Fourier Basis

Definition 4.3 (DTFS Fourier Basis): For the vector space $\mathbb{C}^p$, the DTFS Fourier basis is:

$$\left\{ \boldsymbol{\psi}_0 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}, \boldsymbol{\psi}_1 = \begin{bmatrix} 1 \\ e^{i\frac{2\pi}{p}} \\ e^{i\frac{2\pi}{p}2} \\ \vdots \\ e^{i\frac{2\pi}{p}(p-1)} \end{bmatrix}, \dots, \boldsymbol{\psi}_k = \begin{bmatrix} 1 \\ e^{i\frac{2\pi}{p}k} \\ e^{i\frac{2\pi}{p}2k} \\ \vdots \\ e^{i\frac{2\pi}{p}k(p-1)} \end{bmatrix}, \dots, \boldsymbol{\psi}_{p-1} = \begin{bmatrix} 1 \\ e^{i\frac{2\pi}{p}(p-1)} \\ e^{i\frac{2\pi}{p}2(p-1)} \\ \vdots \\ e^{i\frac{2\pi}{p}(p-1)(p-1)} \end{bmatrix} \right\} \quad (8)$$

To be more concise, for the basis vector $\boldsymbol{\psi}_k$, component n of the vector (starting with $k = 0$ for the first vector and $n = 0$ for the first component) is:

$$(\boldsymbol{\psi}_k)[n] = e^{i\frac{2\pi}{p}nk} = \left(e^{i\frac{2\pi}{p}}\right)^{nk} \quad (9)$$

You can think of this as the following. Our end goal is to write the signal x at each time step, so $x[n]$, as a sum of exponentials. So, two things need to happen. First, we need to index over the different frequencies that make up a signal that is p -periodic, which we keep track of with the k index. And second, we need to do the first step for each timestep of the signal, which we keep track of with the n index.

This is not very easy to see for the first time so let's show what the basis looks like for familiar vector spaces.

Note: The Fourier basis naturally operates with complex exponentials and thus is defined with complex vector spaces, but can (and often will) be applied to real vectors (note that real vector spaces are subspaces of complex vector spaces).

Example 4.1 (DTFS Fourier Basis for $\mathbb{C}^1$, $\mathbb{C}^2$, and $\mathbb{C}^3$):

First, let's start with $\mathbb{C}^1$, the 1D space for complex numbers.

In this case, $p = 1$ so we will only have one 1D basis vector $\boldsymbol{\psi}_0 = \left[e^{i\frac{2\pi}{1}(0)(0)} \right] = [1]$ (just a scalar, which makes sense for 1D space).

For $\mathbb{C}^2$, $p = 2$ and there will be 2 basis vectors $\boldsymbol{\psi}_0$ and $\boldsymbol{\psi}_1$:

$$\boldsymbol{\psi}_0 = \begin{bmatrix} e^{i\frac{2\pi}{2}(0)(0)} \\ e^{i\frac{2\pi}{2}(0)(1)} \end{bmatrix} = \begin{bmatrix} e^{i0} \\ e^{i0} \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad (10)$$

$$\boldsymbol{\psi}_1 = \begin{bmatrix} e^{i\frac{2\pi}{2}(1)(0)} \\ e^{i\frac{2\pi}{2}(1)(1)} \end{bmatrix} = \begin{bmatrix} 1 \\ e^{i\pi} \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \quad (11)$$

For $\mathbb{C}^3$, $p = 3$ and there will be 3 basis vectors $\boldsymbol{\psi}_0$, $\boldsymbol{\psi}_1$, and $\boldsymbol{\psi}_2$:

$$\boldsymbol{\psi}_0 = \begin{bmatrix} e^{i\frac{2\pi}{3}(0)(0)} \\ e^{i\frac{2\pi}{3}(0)(1)} \\ e^{i\frac{2\pi}{3}(0)(2)} \end{bmatrix} = \begin{bmatrix} e^{i0} \\ e^{i0} \\ e^{i0} \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \quad (12)$$

$$\boldsymbol{\psi}_1 = \begin{bmatrix} e^{i\frac{2\pi}{3}(1)(0)} \\ e^{i\frac{2\pi}{3}(1)(1)} \\ e^{i\frac{2\pi}{3}(1)(2)} \end{bmatrix} = \begin{bmatrix} 1 \\ e^{i\frac{2\pi}{3}} \\ e^{i\frac{4\pi}{3}} \end{bmatrix} = \begin{bmatrix} 1 \\ -\frac{1}{2} + i\frac{\sqrt{3}}{2} \\ -\frac{1}{2} - i\frac{\sqrt{3}}{2} \end{bmatrix} \quad (13)$$

$$\boldsymbol{\psi}_2 = \begin{bmatrix} e^{i\frac{2\pi}{3}(2)(0)} \\ e^{i\frac{2\pi}{3}(2)(1)} \\ e^{i\frac{2\pi}{3}(2)(2)} \end{bmatrix} = \begin{bmatrix} 1 \\ e^{i\frac{4\pi}{3}} \\ e^{i\frac{8\pi}{3}} \end{bmatrix} = \begin{bmatrix} 1 \\ -\frac{1}{2} - i\frac{\sqrt{3}}{2} \\ -\frac{1}{2} + i\frac{\sqrt{3}}{2} \end{bmatrix} \quad (14)$$

Note: Phase is 2π -periodic (adding multiples 2π returns the phase to itself) so in the $\mathbb{C}^3$ case, $e^{i\frac{8\pi}{3}} = e^{i(\frac{8\pi}{3} - 2\pi)} = e^{i\frac{2\pi}{3}}$. This will occur frequently with the DTFS Fourier basis vectors.

Both the polar and Cartesian forms are presented, though the polar form tends to be more compact (since the Fourier basis is based on complex exponentials).

Hopefully the pattern for these basis vectors is more clear with these examples and you can see how this would be extended to higher dimensions.

4.1.3 Converting to the DTFS Fourier Basis

Now that we have a definition for the DTFS Fourier basis, we can look to understand how to convert vectors from the standard basis to the DTFS Fourier basis.

Definition 4.4 (DTFS Fourier Basis Representation):

Consider a vector $\mathbf{v} = \begin{bmatrix} v_0 \\ v_1 \\ \vdots \\ v_{p-1} \end{bmatrix} \in \mathbb{C}^p$.

The DTFS Fourier basis representation of $\mathbf{v}$ is $\mathbf{X} = \begin{bmatrix} X_0 \\ X_1 \\ \vdots \\ X_{p-1} \end{bmatrix}$ such that

$$\mathbf{v} = X_0\boldsymbol{\psi}_0 + X_1\boldsymbol{\psi}_1 + \dots + X_{p-1}\boldsymbol{\psi}_{p-1} = \sum_{k=0}^{p-1} X_k\boldsymbol{\psi}_k \quad (15)$$

where $\boldsymbol{\psi}_0, \boldsymbol{\psi}_1, \dots, \boldsymbol{\psi}_{p-1}$ are the DTFS Fourier basis vectors.

Basically, $\mathbf{v}$ can be written as a linear combination of the DTFS Fourier basis vectors $\boldsymbol{\psi}_k$; this is precisely what a change of basis means!

Let's do an example of this for $\mathbb{C}^2$.

Example 4.2 (DTFS Fourier Basis Representation in $\mathbb{C}^2$):

Consider the vector $\mathbf{v} = \begin{bmatrix} -2 \\ 3 \end{bmatrix}$. We want to find the DTFS Fourier basis representation $\mathbf{X} = \begin{bmatrix} X_0 \\ X_1 \end{bmatrix}$.

We know the basis vectors for $\mathbb{C}^2$ from Example 4.1 and so we can set up the change of basis equation from Definition 4.4:

$$\mathbf{v} = X_0 \boldsymbol{\psi}_0 + X_1 \boldsymbol{\psi}_1 \quad (16)$$

$$\begin{bmatrix} -2 \\ 3 \end{bmatrix} = X_0 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + X_1 \begin{bmatrix} 1 \\ -1 \end{bmatrix} \quad (17)$$

$$\begin{bmatrix} -2 \\ 3 \end{bmatrix} = \begin{bmatrix} X_0 + X_1 \\ X_0 - X_1 \end{bmatrix} \quad (18)$$

$$X_0 + X_1 = -2 \quad (19)$$

$$X_0 - X_1 = 3 \quad (20)$$

Solving the system of equations, we can find that $X_0 = \frac{1}{2}$ and $X_1 = -\frac{5}{2}$ so the Fourier basis representation is:

$$\mathbf{X} = \begin{bmatrix} \frac{1}{2} \\ -\frac{5}{2} \end{bmatrix} \quad (21)$$

This basis representation can be represented graphically. Here are the $\mathbb{C}^2$ basis vectors represented in the 2D Cartesian plane:

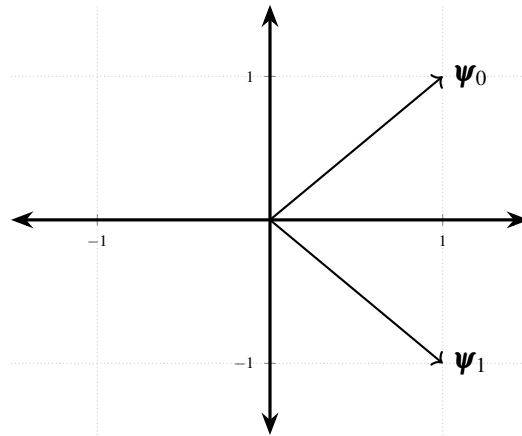


Figure 3: 2D DTFS Fourier Basis Vectors

Here is the construction of $\mathbf{v} = \begin{bmatrix} -2 \\ 3 \end{bmatrix}$ using $X_0\psi_0$ and $X_1\psi_1$:

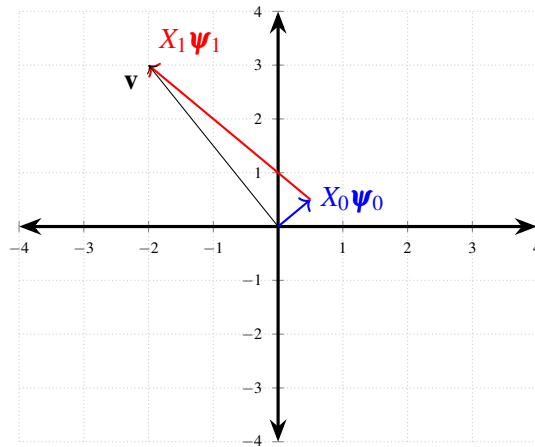


Figure 4: Vector $\mathbf{v} = \begin{bmatrix} -2 \\ 3 \end{bmatrix}$ in the 2D DTFS Fourier Basis

Suppose in this example, we generalized the vector to $\mathbf{v} = \begin{bmatrix} v_0 \\ v_1 \end{bmatrix}$. Performing the same operations with this generalized vector will yield:

$$X_0 = \frac{1}{2}(v_0 + v_1) \quad (22)$$

$$X_1 = \frac{1}{2}(v_0 - v_1) \quad (23)$$

Notice that we can write these using inner products of the vector $\mathbf{v}$ with the basis vectors $\boldsymbol{\psi}_0$ and $\boldsymbol{\psi}_1$:

$$X_0 = \frac{1}{2} \begin{bmatrix} v_0 & v_1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{p} \mathbf{v}^\top \boldsymbol{\psi}_0^* = \frac{1}{p} \langle \mathbf{v}, \boldsymbol{\psi}_0 \rangle \quad (24)$$

$$X_1 = \frac{1}{2} \begin{bmatrix} v_0 & v_1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{p} \mathbf{v}^\top \boldsymbol{\psi}_1^* = \frac{1}{p} \langle \mathbf{v}, \boldsymbol{\psi}_1 \rangle \quad (25)$$

where $p = 2$ is the dimensionality of the vectors (2 in this case).

This idea, that the change of basis can be done by using inner products of the vector being transformed with the new basis vectors, can be extended to higher dimensions as well and is actually true of any orthogonal basis. Thus, we will show that the DTFS Fourier basis is orthogonal and show the importance of this.

Note: Including the complex conjugates doesn't matter here, but they will for higher dimensions when the DTFS Fourier basis vectors start to include complex numbers (and thus we use the complex inner product).

4.1.4 Orthogonality of the DTFS Fourier Basis

To show orthogonality of the DTFS Fourier basis, we need to show:

$$\langle \boldsymbol{\psi}_k, \boldsymbol{\psi}_l \rangle = 0 \quad (\text{for } k \neq l) \quad (26)$$

Let's start by expanding $\langle \boldsymbol{\psi}_k, \boldsymbol{\psi}_l \rangle$ using summation notation:

$$\langle \boldsymbol{\psi}_k, \boldsymbol{\psi}_l \rangle = \boldsymbol{\psi}_k^\top \boldsymbol{\psi}_l^* \quad (27)$$

$$= \sum_{m=0}^{p-1} e^{i\frac{2\pi}{p}km} e^{-i\frac{2\pi}{p}lm} \quad (28)$$

$$= \sum_{m=0}^{p-1} e^{i\frac{2\pi}{p}(k-l)m} \quad (29)$$

For the case of $k \neq l$:

$$\sum_{m=0}^{p-1} e^{i\frac{2\pi}{p}(k-l)m} = \frac{1 - (e^{i\frac{2\pi}{p}(k-l)})^p}{1 - e^{i\frac{2\pi}{p}(k-l)}} \quad (30)$$

$$= \frac{1 - e^{i2\pi(k-l)}}{1 - e^{i\frac{2\pi}{p}(k-l)}} \quad (31)$$

where we use the finite geometric series formula.

Geometric Series Formula:

$$\sum_{k=0}^{p-1} x^k = \frac{1 - x^p}{1 - x} \quad (32)$$

Since $(k-l)p$ is an integer, $e^{i2\pi(k-l)p} = 1$, which means that for $k \neq l$, $\langle \boldsymbol{\psi}_k, \boldsymbol{\psi}_l \rangle = 0$.

Thus, we have shown that the Fourier basis vectors are orthogonal.

What about the case where $k = l$? In vector form, this corresponds to:

$$\langle \boldsymbol{\psi}_k, \boldsymbol{\psi}_k \rangle = \|\boldsymbol{\psi}_k\|^2 \quad (33)$$

Thus, this case will tell us how normalized the DTFS Fourier basis vectors are. Using the same calculation:

$$\sum_{m=0}^{p-1} e^{i\frac{2\pi}{p}(k-l)m} = \sum_{m=0}^{p-1} e^{i\frac{2\pi}{p}(0)m} \quad (34)$$

$$= \sum_{m=0}^{p-1} 1 \quad (35)$$

$$= p \quad (36)$$

Thus,

$$\langle \boldsymbol{\psi}_k, \boldsymbol{\psi}_k \rangle = \|\boldsymbol{\psi}_k\|^2 = p \quad (37)$$

To summarize:

$$\langle \boldsymbol{\psi}_k, \boldsymbol{\psi}_l \rangle = \begin{cases} p & k = l \\ 0 & k \neq l \end{cases} \quad (38)$$

4.1.5 Validity of the DTFS Fourier Basis

Recall that for a set of vectors to be a valid basis, the vectors in the set must be linearly independent and must span the vector space they are part of.

Since the set of DTFS Fourier basis vectors $\{\boldsymbol{\psi}_0, \dots, \boldsymbol{\psi}_{p-1}\}$ is orthogonal (and nonzero), the set of vectors must be linearly independent. Additionally, since $\mathbb{C}^p$ has dimension p , the fact that $\{\boldsymbol{\psi}_0, \dots, \boldsymbol{\psi}_{p-1}\}$ is a set of p linearly independent vectors in $\mathbb{C}^p$ means that the DTFS Fourier basis spans $\mathbb{C}^p$.

Thus, the DTFS Fourier basis $\{\boldsymbol{\psi}_0, \dots, \boldsymbol{\psi}_{p-1}\}$ is a valid basis for $\mathbb{C}^p$.

4.1.6 Discrete-Time Fourier Series (DTFS) Formula

Now that we have established that the DTFS Fourier basis is a valid, orthogonal basis, we are ready to return to discrete-time signals and a **very important** concept (used all the time in signal processing): the Discrete-Time Fourier Series (DTFS) formula itself. First we present the formulas and then show why they're true.

Definition 4.5 (Discrete-Time Fourier Series (DTFS) Formula):

Suppose we have a discrete-time signal x that is p -periodic.

There exists a unique set of p DTFS coefficients for $x[n]$, denoted X_k for $k = 0, 1, \dots, p-1$.

The formula to convert $x[n]$ to X_k is:

$$X_k = \frac{1}{p} \sum_{n=0}^{p-1} x[n] e^{-i\frac{2\pi}{p}nk} = \frac{1}{p} \sum_{n \in \langle p \rangle} x[n] e^{-i\frac{2\pi}{p}nk} \quad (39)$$

This part of the DTFS is often referred to as the **DTFS Analysis** equation (because we are decomposing the signal $x[n]$ into the coefficients of each frequency component, ie. each of the Fourier basis vectors). Moreover, $\langle p \rangle$ is defined as a set of p contiguous integers. For example $\{0, 1, \dots, p-1\}$ or $\{1, 2, \dots, p\}$. This sum is valid for any such interval $\langle p \rangle$ because of the periodic nature of DTFS. We typically use $\{0, 1, \dots, p-1\}$ but using some other set may be more helpful in some cases.

Since the DTFS corresponds to a one-to-one mapping with a vector change of basis, there is a one-to-one mapping between each discrete-time signal x and its DTFS coefficients X_k . Thus, we can also convert from X_k back to $x[n]$; this is called the Inverse Discrete-Time Fourier Series (IDTFS).

Definition 4.6 (Inverse Discrete-Time Fourier Series (DTFS) Formula):

Suppose we have a set of p DTFS coefficients X_k for $k = 0, 1, \dots, p-1$.

There exists a unique p -periodic discrete-time signal that has these DTFS X_k coefficients, denoted x .

The formula to convert X_k to the value the signal takes at time n , $x[n]$, is:

$$x[n] = \sum_{k=0}^{p-1} X_k e^{i\frac{2\pi}{p}nk} = \sum_{k \in \langle p \rangle} X_k e^{i\frac{2\pi}{p}nk} \quad (40)$$

This part of the DTFS is often referred to as the **DTFS Synthesis** equation (because we are composing the signal at all indices with the coefficients of its frequency components and the Fourier basis vectors).

Why is this true?

To start out, let's convert $x[n]$ to its vector representation:

$$\mathbf{x} = \begin{bmatrix} x[0] \\ x[1] \\ \vdots \\ x[p-1] \end{bmatrix} \quad (41)$$

In this representation, we can rewrite the DTFS synthesis equation. Recall that:

$$(\boldsymbol{\psi}_k)[n] = e^{i\frac{2\pi}{p}nk} \quad (42)$$

Thus,

$$x[n] = \sum_{k=0}^{p-1} X_k e^{i\frac{2\pi}{p}nk} = \sum_{k=0}^{p-1} X_k (\boldsymbol{\psi}_k)[n] \quad (43)$$

By stacking $x[n]$ and $(\boldsymbol{\psi}_k)[n]$ into vectors:

$$\mathbf{x} = \sum_{k=0}^{p-1} X_k \boldsymbol{\psi}_k \quad (44)$$

Note: This is the exact same as Equation 15! The vector change of basis equation from earlier is the DTFS synthesis equation, just using the vector representation of discrete-time signals.

Now, our goal is to extract X_k . To do so, we will make use of the fact that the DTFS Fourier basis is orthogonal.

Let's calculate $\langle \mathbf{x}, \boldsymbol{\psi}_l \rangle$:

$$\langle \mathbf{x}, \boldsymbol{\psi}_l \rangle = \left\langle \sum_{k=0}^{p-1} X_k \boldsymbol{\psi}_k, \boldsymbol{\psi}_l \right\rangle \quad (45)$$

We can use the linearity of the inner product to bring it inside the summation:

$$\left\langle \sum_{k=0}^{p-1} X_k \boldsymbol{\psi}_k, \boldsymbol{\psi}_l \right\rangle = \sum_{k=0}^{p-1} X_k \langle \boldsymbol{\psi}_k, \boldsymbol{\psi}_l \rangle \quad (46)$$

We know from Equation 38 that:

$$\langle \boldsymbol{\psi}_k, \boldsymbol{\psi}_l \rangle = \begin{cases} p & k = l \\ 0 & k \neq l \end{cases} \quad (47)$$

Thus, the only nonzero term in the summation is the one with $k = l$, which means:

$$\sum_{k=0}^{p-1} X_k \langle \boldsymbol{\psi}_k, \boldsymbol{\psi}_l \rangle = X_l \langle \boldsymbol{\psi}_l, \boldsymbol{\psi}_l \rangle = X_l \cdot p \quad (48)$$

Thus, $X_l = \frac{1}{p} \langle \mathbf{x}, \boldsymbol{\psi}_l \rangle$.

Replacing l with k (this is just an index) and expanding $\langle \mathbf{x}, \boldsymbol{\psi}_k \rangle$ into a summation, we get:

$$X_k = \frac{1}{p} \langle \mathbf{x}, \boldsymbol{\psi}_k \rangle = \frac{1}{p} \mathbf{x}^\top \boldsymbol{\psi}_k^* = \frac{1}{p} \sum_{n=0}^{p-1} x[n] e^{-i\frac{2\pi}{p}nk} \quad (49)$$

which is exactly the DTFS analysis equation (Equation 39)!

4.1.7 Frequency Interpretation of the DTFS

You may recall that we mentioned how the Fourier basis emphasizes the frequency of the data. This mostly comes from the discrete-time signal concept in which there is a sense of time (through samples).

We can see this by looking at the general form for the DTFS Fourier basis vectors:

$$\boldsymbol{\psi}_k = \begin{bmatrix} 1 \\ e^{j\frac{2\pi}{p}k} \\ e^{j\frac{2\pi}{p}2k} \\ \vdots \\ e^{j\frac{2\pi}{p}k(p-1)} \end{bmatrix} \quad (50)$$

Notice that the exponent increases by $\frac{2\pi}{p}k$ with each timestep. Thus, the frequency associated vector $\boldsymbol{\psi}_k$ is $\omega_k = \frac{2\pi}{p}k$.

Let's look at the 2D example:

Example 4.3 (Frequency Interpretation for $\mathbb{C}^2$ DTFS Fourier Basis Vectors):

Here is the stem plot for $\boldsymbol{\psi}_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$:

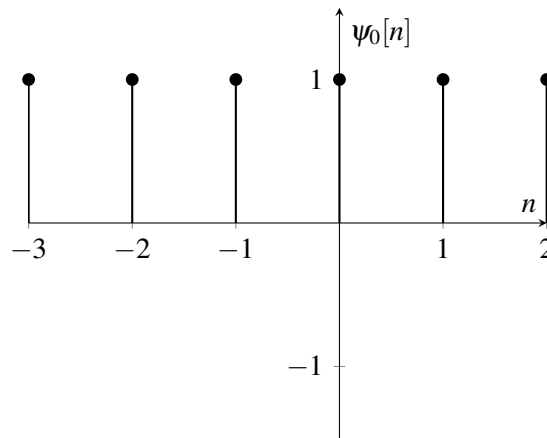


Figure 5: Stem Plot for $\boldsymbol{\psi}_0 \in \mathbb{C}^2$

Notice that the signal stays constant for all samples. Since the signal does not change, we consider it to have a frequency of $\omega_0 = 0$ (also called DC).

Here is the stem plot for $\boldsymbol{\psi}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$:

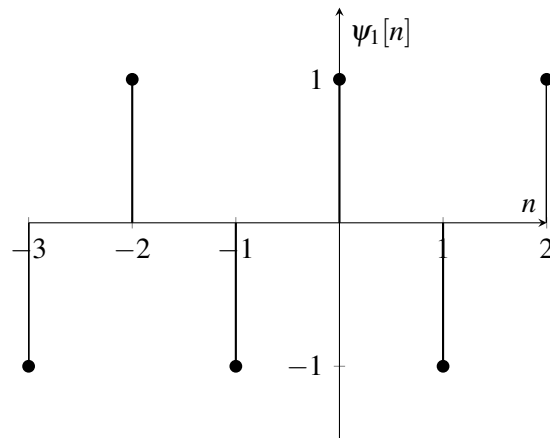


Figure 6: Stem Plot for $\psi_1 \in \mathbb{C}^2$

Notice that the signal cycles up and down every sample (it uses exactly 2 samples per cycle). Thus, we consider it to have a frequency of $\omega_1 = \frac{2\pi}{2} = \pi$. (Note: This is actually the highest frequency for discrete time; we can't be any faster than cycling every 2 samples.)

Here is the stem plot for $\mathbf{v} = \begin{bmatrix} -2 \\ 3 \end{bmatrix}$:

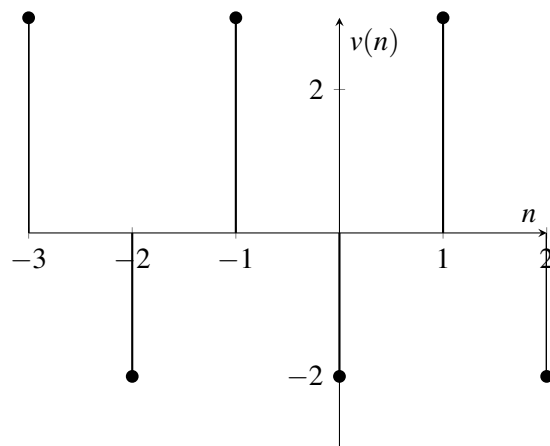


Figure 7: Stem Plot for $\mathbf{v} = \begin{bmatrix} -2 \\ 3 \end{bmatrix} \in \mathbb{C}^2$

We know (from Example 4.2) that the Fourier representation of this vector is:

$$\mathbf{X} = \begin{bmatrix} \frac{1}{2} \\ -\frac{5}{2} \end{bmatrix} \quad (51)$$

Visibly, the discrete-time signal waveform is more like ψ_1 than ψ_0 because the signal jumps by 5 up or down each sample; thus, the signal is mostly *high frequency*, which matches the observation of Fourier coefficient $X_1 = -\frac{5}{2}$ (the $\omega = \pi$ coefficient) being higher in magnitude than $X_0 = \frac{1}{2}$ (the $\omega = 0$, or DC, component).

4.1.8 Discrete Fourier Transform (DFT)

Finally, another Fourier Transform that deals with periodic discrete-time signals which consistently shows up in Signal Processing resources and textbooks is the Discrete Fourier Transform (DFT). For example, the Python library `numpy` implements its Fourier Transform functions using the DFT. It is essentially the same thing as the DTFS but it has a scaling factor of $\frac{1}{p}$, namely, $\phi_k = \frac{1}{p}\psi_k$.

$$\left\{ \phi_0 = \frac{1}{p} \begin{bmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}, \phi_1 = \frac{1}{p} \begin{bmatrix} 1 \\ e^{i\frac{2\pi}{p}} \\ e^{i\frac{2\pi}{p}2} \\ \vdots \\ e^{i\frac{2\pi}{p}(p-1)} \end{bmatrix}, \dots, \phi_k = \frac{1}{p} \begin{bmatrix} 1 \\ e^{i\frac{2\pi}{p}k} \\ e^{i\frac{2\pi}{p}2k} \\ \vdots \\ e^{i\frac{2\pi}{p}k(p-1)} \end{bmatrix}, \dots, \phi_{p-1} = \frac{1}{p} \begin{bmatrix} 1 \\ e^{i\frac{2\pi}{p}(p-1)} \\ e^{i\frac{2\pi}{p}2(p-1)} \\ \vdots \\ e^{i\frac{2\pi}{p}(p-1)(p-1)} \end{bmatrix} \right\} \quad (52)$$

Lastly, we get the following equations.

The **DFT formula (or analysis equation)** is:

$$X_k = \sum_{n=0}^{p-1} x[n] e^{-i\frac{2\pi}{p}nk} = \sum_{n \in \langle p \rangle} x[n] e^{-i\frac{2\pi}{p}nk} \quad (53)$$

The **Inverse DFT formula (or synthesis equation)** is:

$$x[n] = \frac{1}{p} \sum_{k=0}^{p-1} X_k e^{i\frac{2\pi}{p}nk} = \frac{1}{p} \sum_{k \in \langle p \rangle} X_k e^{i\frac{2\pi}{p}nk} \quad (54)$$

4.1.9 DTFS/DFT Applications

Throughout this note, we have discussed a lot about what the DTFS (and DFT) are, the math behind them, and how they reveal the frequency information of a discrete-time signal.

But where is the DTFS/DFT actually used? Why is it so important?

To start off, the concept of Fourier analysis is not limited to periodic discrete-signals. It can be applied to both continuous signals and discrete-time signals in the form of Fourier series for periodic signals and Fourier transforms for all signals. This table shows the possible cases (for different signal types):

Signal Type	Periodic	Aperiodic
Discrete	Discrete-Time Fourier Series (DTFS)	Discrete-Time Fourier Transform (DTFT)
Continuous	Continuous-Time Fourier Series (CTFS)	Continuous-Time Fourier Transform (CTFT)

Note: Even though the DFT is called a "Fourier transform," it is actually a special form of the DTFS and thus is more of a Fourier series. The DFT is much more similar to the DTFS than the DTFT!

One application of the DTFS/DFT that you have undoubtedly encountered (though you may not have known that it used Fourier transforms) is image compression. While you could do this with either the DTFS or DFT, computation packages like `numpy` use the DFT so that is what will be used through the rest of the example.

Consider the following image (of a chip wafer)¹:

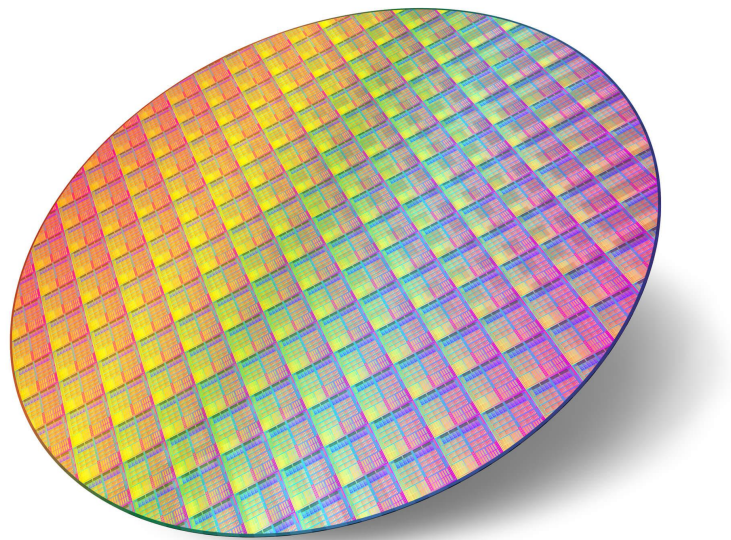


Figure 8: Wafer Image

You may be familiar with the fact that images can be separated into red, green, and blue components (RGB). Here are the RGB component plots of the wafer image:

¹Credit to Electrical Engineering Stack Exchange for the image.

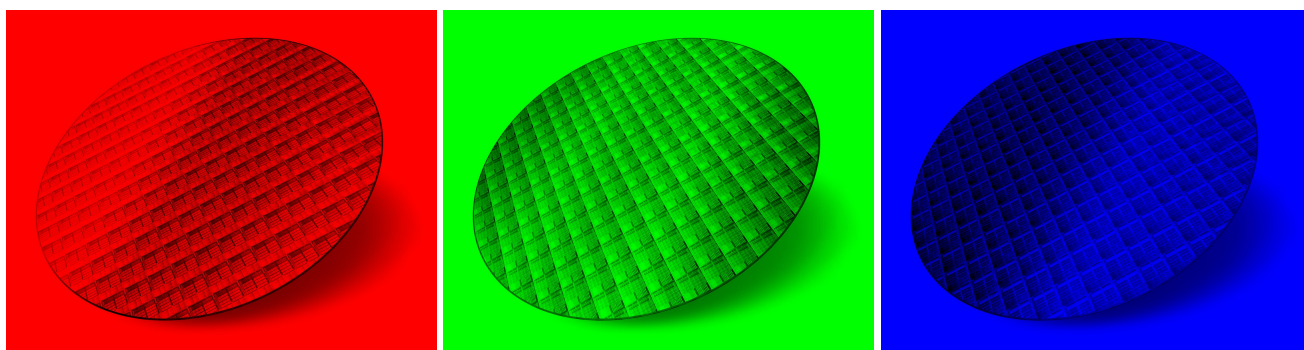


Figure 9: RGB Sub-Images

Without thinking about file size, if you wanted to store an image, the standard way to do so would be to store the RGB components of each pixel in the image.

The wafer image is 2000×1600 pixels for a total of 3.2 million pixels. Storing RGB components for each of these pixels would take 9.6 million numbers (the tiff file format size for this image, which has no compression, is 9.6 MB large); that's a lot of data! We know image file sizes are not actually this large; the JPEG compression of this image is only 493 KB. Why is this so?

The key here is that most images are not random; if one pixel has certain RGB components, the nearby pixels probably have very similar components. This idea that the RGB components of pixels do not change quickly means that most images are dominated by *low frequencies*.

To see this, here are the plots of the 2D DFT of each of the RGB components of the wafer image:

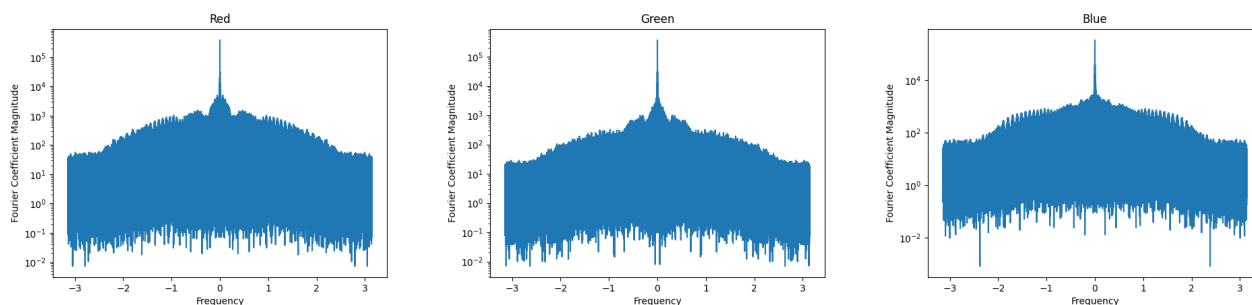


Figure 10: Plots of Fourier Coefficients for RGB Sub-Images

The center is the lowest frequency (DC) and the frequency increases as you move away from the center. Each of the plots peak at DC and the DFT coefficient tends to decrease as the frequency increases.

We can compress the image by effectively removing some of these higher frequency coefficients with low values, which can be done by thresholding the coefficients (setting coefficients below some chosen threshold value to 0). After removing 56% of the total RGB coefficients, we are left the following image:

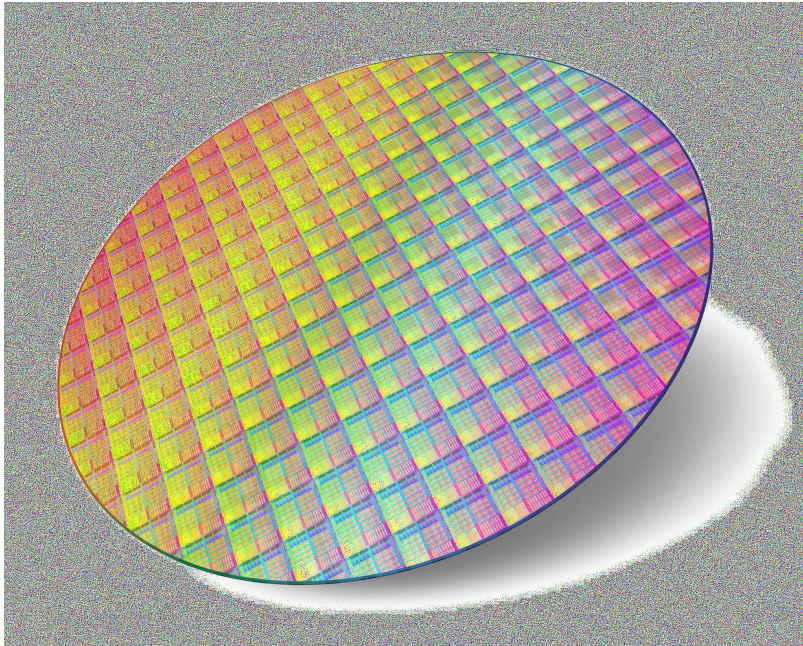


Figure 11: Wafer Image with Thresholded Coefficients

Here is a side-by-side comparison with the original image:

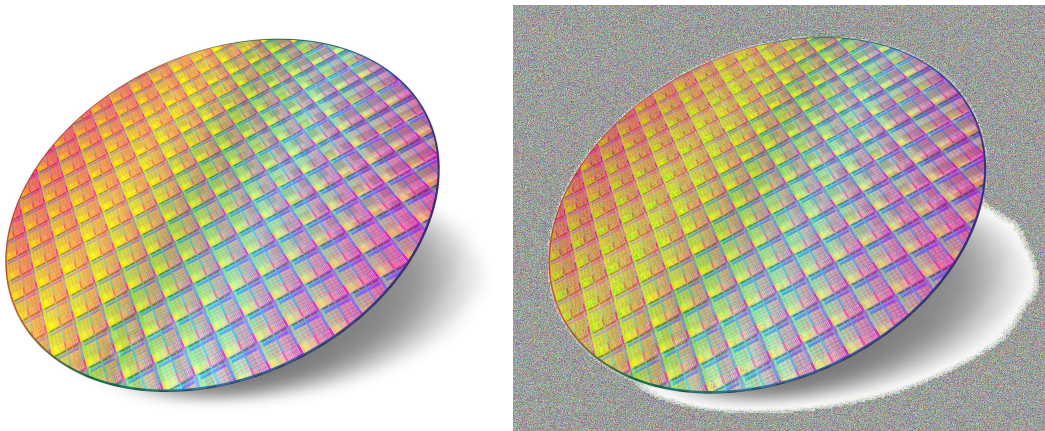


Figure 12: Wafer Image Comparison

Pretty good for using less than half the Fourier coefficients, right? The reconstructed image has obvious artifacts (the background looks noisy), but the important details are still there (the wafer itself is clearly visible in pretty decent quality); definitely not just half the image!

To fix the artifacts (and get much better compression), we would need more developed compression algorithms like JPEG, which make use of more signal processing techniques that you would learn about in classes like EE 123! Pretty much all of this processing happens post-DFT due to the usefulness of the frequency spectrum of data. By understanding the DFT, you have an incredibly useful mathematical tool that can be used for many awesome applications!